Stochastic Control

Homework Project 2

Reinforcement Learning with Policy Gradient

Ioannis Dimoulios

Mat. Num. 470364

Winter Semester 2024

Problem 1

a) Simple algebraic manipulation yields

$$\xi(t_{k+1}) = \underbrace{\xi(t_k) + 0.2\delta t \begin{bmatrix} -\sin\left(\frac{t_k}{50}\right) \\ +\cos\left(\frac{t_k}{50}\right) \end{bmatrix}}_{f_1(\xi(t_k), t_k)} + \begin{bmatrix} w_1(t_k) \\ w_2(t_k) \end{bmatrix}$$

Now taking into account that $w \sim \text{Laplace}(0, \sigma)$, we get that

$$\rho_1(\xi(t_{k+1})|\xi(t_k), t_k) \sim \text{Laplace}(f_1(\xi(t_k), t_k), \sigma),$$

which holds per coordinate.
b) In similar fashion

$$\zeta(t_k) = \xi(t_k) + \begin{bmatrix} v_1(t_k) \\ v_2(t_k) \end{bmatrix}$$
$$\rho(\zeta(t_k)|\xi(t_k)) \sim \mathcal{U}(\xi(t_k) - \varepsilon, \xi(t_k) + \varepsilon),$$

which again holds per coordinate.

Problem 2

a) Monte Carlo integration conveys the simple idea, that if we cannot analytically calculate the expected value of a random variable, we can just draw various realizations of the random variable. Then, their average will converge to the expected value as the number of samples grows.
Importance sampling on the other hand says that if we do cannot draw samples from this particular distribution $\rho(\cdot)$, but we have a good enough approximation distribution $\mu(\cdot)$ (importance density), then we may as well draw samples from the approximation. In this case, as long as we take the weighted mean of the samples the estimate will still be a good approximation of the expected value. Each weight corresponds to the value

$$w^{(i)} = \frac{1}{N} \frac{\rho(\xi^{(i)})}{\mu(\xi^{(i)})},$$

where N is the number of samples and $\xi^{(i)}$ the i-th sample.

b) State estimation essentially requires that we find the unknown state which evolves over time when we know that each moment it follows a particular distribution, and we have prior information of the output of the system, i.e. noisy measurements of a function of the state. To do this, we use the particle filter which provides a systematic way to sample a good enough approximation of the distribution of the state, which evolve over time employing the system dynamics, and calculate reliable weights to get a weighted average of the samples which in theory should be a good approxiamation of the current state.

// TODO: explain in detail each step of the particle filter.

c) To begin with, x_k corresponds to $\xi(t_k)$. Note that there is no input, i.e. the system is autonomous.
Then, z_k corresponds to $\zeta(t_k)$.

d) The obvious way to choose $q(\cdot)$ is

$$q(x_k^{(i)}|x_{k-1}^{(i)}, u_{k-1}) = \rho(x_k^{(i)}|x_{k-1}^{(i)}, u_{k-1}).$$

This simplifies the calculation of the next batch of weights, as

$$w_k^{(i)} = \rho(z_k|x_k^{(i)})w_{k-1}^{(i)}.$$

An alternative would be to simply let $q(\cdot)$ be a Gaussian distribution. In that sense,

$$q(x_k^{(i)}|x_{k-1}^{(i)}, u_{k-1}) \sim \mathcal{N}(x_k^{(i)}; f(x_{k-1}^{(i)}, u_{k-1}), \Sigma_k),$$

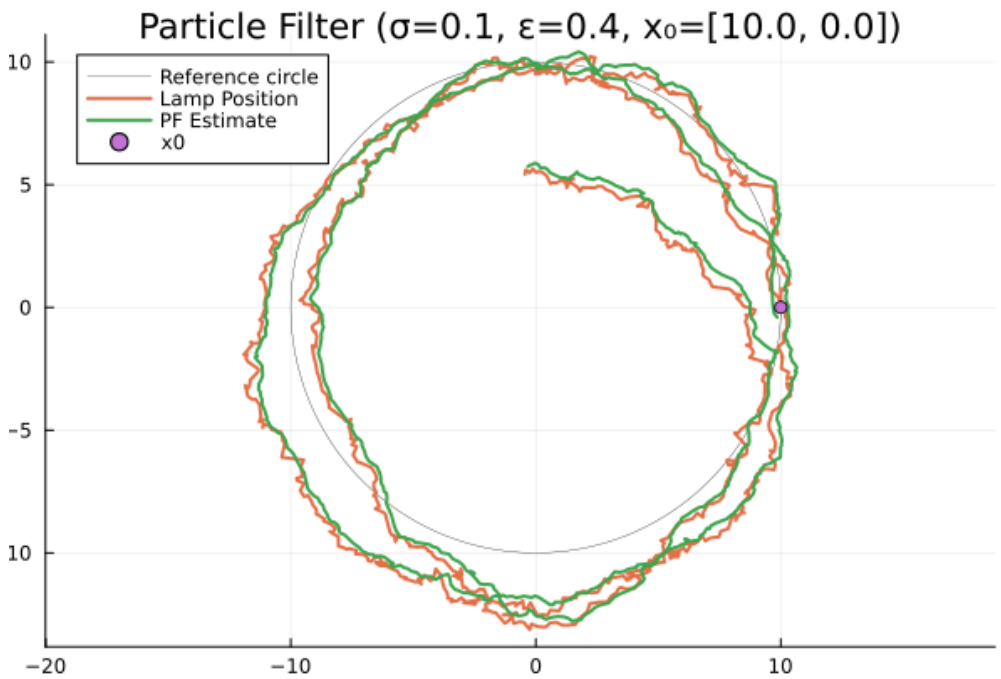
where $f(\cdot)$ is the system dynamics function excluding the non-predictable factor and Σ_k is the to be determined covariance matrix.

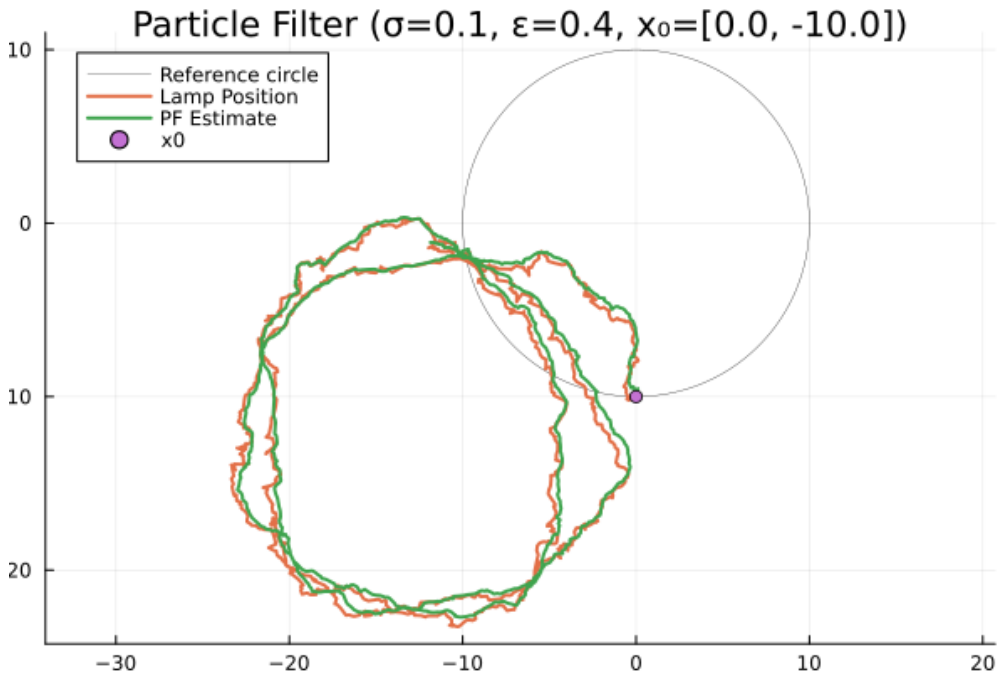
Problem 3

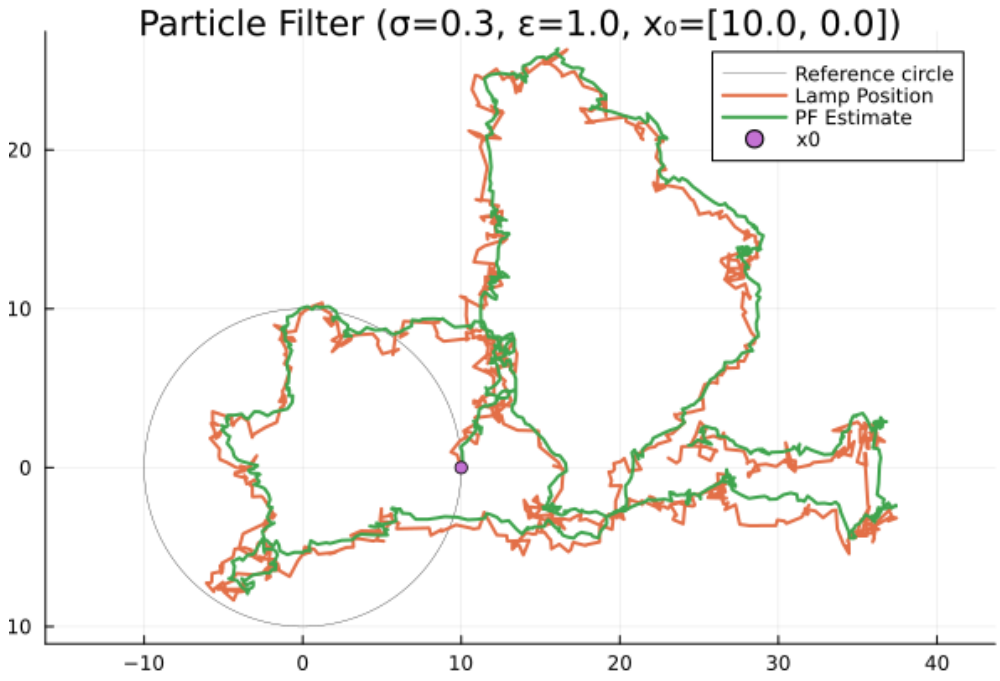
a) Pretty much done already in Problem 1.

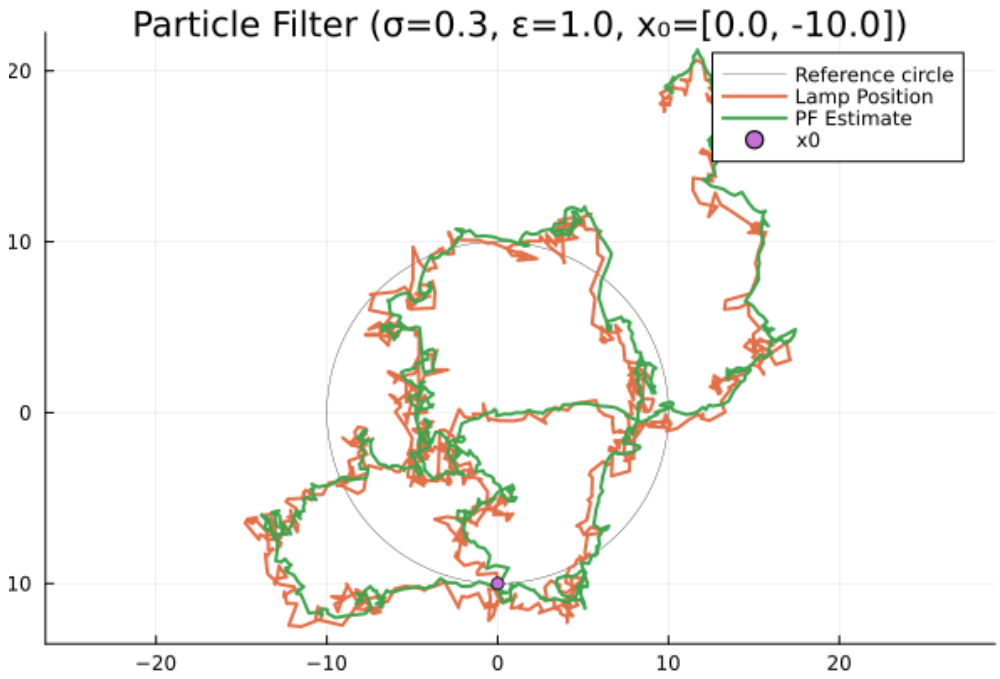
b) We chose the first option outlined in 2d.

c) See source code, namely file `src/particle_filter.jl`




Particle Filter (σ=0.1, ε=0.4, x₀=[0.0, -10.0])
Reference circle
Lamp Position
PF Estimate
x0
10
0
10
20
−30
−20
−10
0
10
20


Particle Filter (σ=0.3, ε=1.0, x₀=[10.0, 0.0])
Reference circle
Lamp Position
PF Estimate
x0
20
10
0
10
−10
0
10
20
30
40



Problem 4

a) In similiar fashion to the discretization in Problem 1,

$$\begin{aligned} \frac{\mathrm{d}\theta}{\mathrm{d}t} &= u \\ \implies \theta_{k+1} - \theta_k &= u_k \delta t \\ \implies \theta_{k+1} &= \theta_k + u_k \delta t. \end{aligned}$$

b) We basically want $u \propto \hat{\xi}_k - \theta_k$ where $\hat{\xi}_k$ is the calculated estimate for ξ_k provided by the particle filter and simultaneously $\|u\|_2 \leq 1$. Hence, simply

$$u = \frac{\hat{\xi}_k - \theta_k}{\max(\|\hat{\xi}_k - \theta_k\|, 1)}.$$

c) See source code, namely file `src/source_seeking.jl`.

d) It works quite well. Maybe the maximum magnitude of the velocity is a little too big and the robot sometimes unnecessarily jiggles trying to mimic the minor displacements of the lamp due to the disturbances in its dynamics.

We have lowered the maximum speed (norm of velocity) of the robot, so that its graph is a little bit smoother.
See the included `.gif` files for the moving plots.